CampusLab

Gestión de reservas académicas y preparación de laboratorios

Experiencia 2 · Desarrollo Cloud Native I

José Muñoz Reyes
Sebastián Santander Aravena



El caso CampusLab

Reservar un equipo también exige coordinar su preparación y devolución.

Necesidad

Controlar disponibilidad y conocer el estado de cada solicitud.

Solución

Una reserva conecta al estudiante con el técnico y los servicios de preparación y notificación.

Usuarios y permisos

01 **Estudiante**

Solicita recursos y consulta sus propias reservas y comprobantes.

02 **Técnico**

Aprueba solicitudes, gestiona la preparación y registra la devolución.

03 **Administrador**

Gestiona recursos y administra la mensajería.

Microsoft Entra ID autentica · El backend verifica los permisos

Arquitectura de la solución

Acceso

Angular + MSAL
Microsoft Entra ID

Frontend local
Identidad en Azure

Servicios en AWS

API Gateway
BFF y microservicios

JWT en cada petición
Contenedores en EC2

Datos y eventos

PostgreSQL RDS
RabbitMQ

Persistencia cloud
Procesamiento asíncrono

La petición HTTP atiende la reserva; los eventos activan las tareas posteriores.

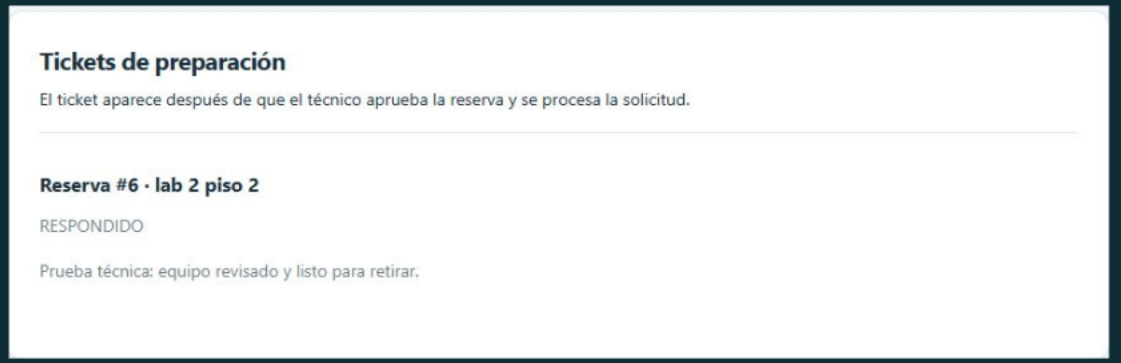
Recorrido de una reserva

1 **Solicitud del estudiante**
2 **Aprobación del técnico**
3 **Ticket de preparación**
4 **Uso y devolución**

El laboratorio corresponde
al recurso seleccionado.

Stock de la prueba: 10, 9 y 10





Reserva y ticket registrados en AWS

Tres tareas mediante RabbitMQ

Cola	Responsabilidad
q.cmd.email	Enviar la notificación por correo
q.cmd.prep	Crear el ticket de preparación
q.cmd.voucher	Generar los comprobantes PDF

direct

Clave exacta

topic

Patrones de enrutamiento

2 nodos

Clúster en EC2

Organización de la implementación

01 **Configuración centralizada**

RabbitTopology y propiedades definen nombres y beans Queue, Exchange y Binding.

02 **Lógica de negocio separada**

ReservaService registra la operación y el outbox gestiona la publicación.

03 **Consumidores por dominio**

EmailConsumer, PreparationConsumer y VoucherConsumer procesan cada tarea.

Qué ocurre cuando falla un mensaje

ACK

Procesamiento correcto
El consumidor confirma el mensaje.

Reintentos

Fallo transitorio
Hasta tres intentos de procesamiento.

DLQ

Mensaje inválido o intentos agotados
NACK lo envía a su cola de errores.

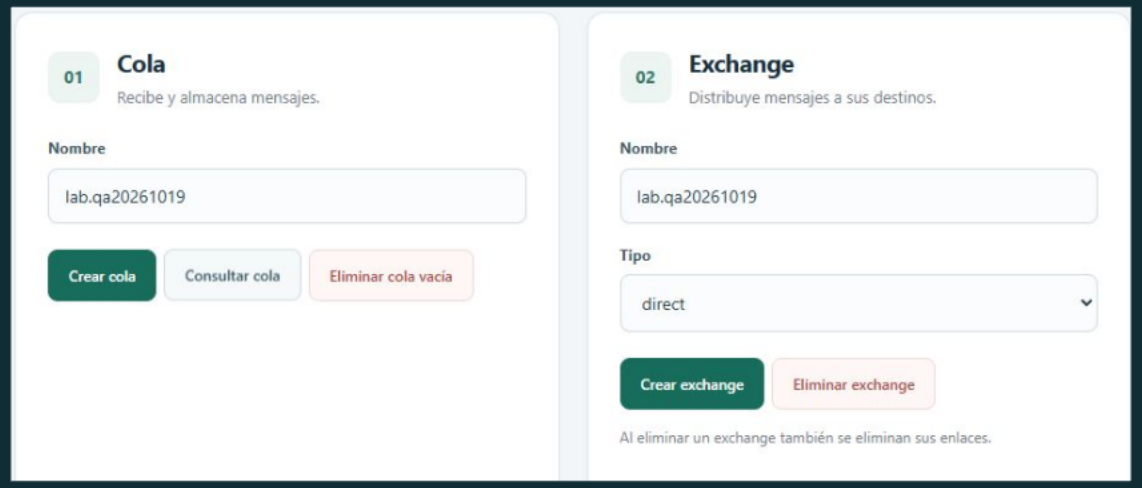
Cada cola principal tiene su propia DLQ para conservar y revisar los fallos.

Administración desde Angular

Crear, consultar y eliminar

API REST protegida
Colas, exchanges y bindings

RabbitAdminService
Encapsula las operaciones AMQP



Validación de nombres lab.* y tipos · Acceso solo para administradores

Evidencias para la demostración

01 Identidad y acceso

Login con Entra ID, consumo de API Gateway y rechazo de peticiones no autorizadas.

02 Reserva y mensajería

Reserva, ticket, PDF y tres DLQ con eventId y motivo de rechazo.

03 Administración y código

Creación y eliminación de recursos de prueba, con fuentes en GitHub.

Evidencia registrada en AWS · 19 y 22 de octubre de 2026